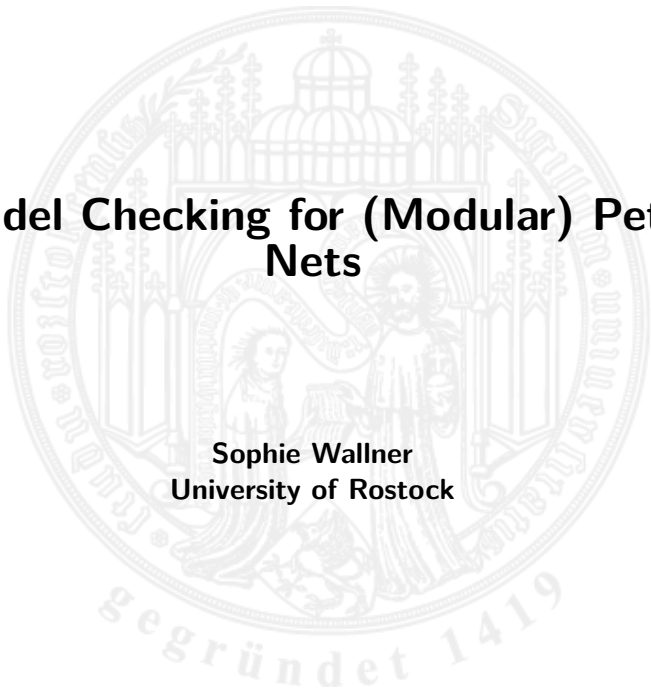


# Model Checking for (Modular) Petri Nets

Sophie Wallner  
University of Rostock



# **1. Model Checking**

## Computer-aided Verification Method [CES86]

→ Evaluate behavior of a system

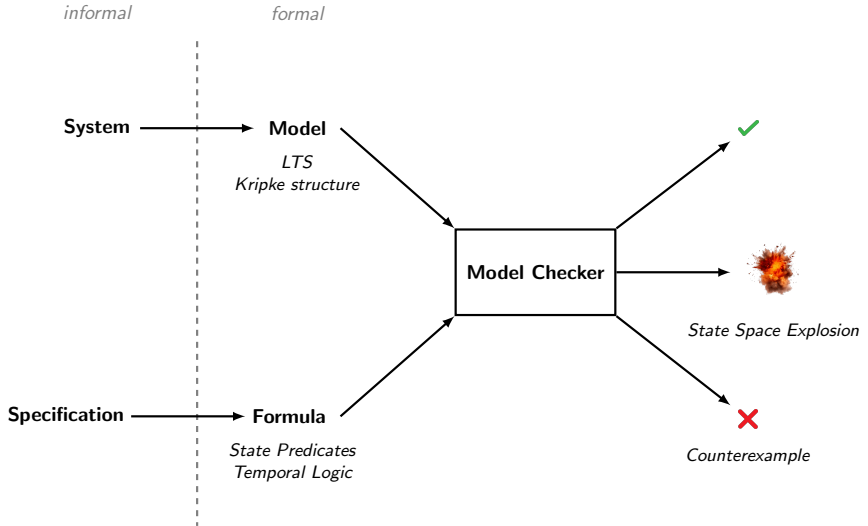
≠ Testing

### Fields of Application

- Safety-critical systems (traffic, aviation, medical devices, etc.)
- Mission-critical systems (extensive testing not possible, e.g., spaceflight)



# Model Checking



## **2. Petri Nets**

Modeling formalism for discrete Systems

Causality and concurrency can be easily illustrated

**Definition:** Petri Net, Marking [DR96]

A *Petri net* is a tuple  $N = [P, T, f, w, m_0]$ :

- $P$  – finite set of *places*
- $T$  – finite set of *transitions* with  $P \cap T = \emptyset$
- $f \subseteq (P \times T) \cup (T \times P)$  – *flow relation*
- $w : f \rightarrow \mathbb{N} \setminus \{0\}$  – *weight function*
- $m_0$  is the initial marking

A *marking* is a mapping  $m : P \rightarrow \mathbb{N}$ .

## Petri Net Structure

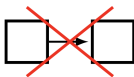
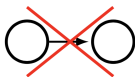
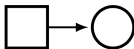
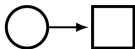
**Places** – State parameters  
passive Components



**Transitions** – Actions  
active Components



**Directed (weighted) edges** – Connection of places and transitions



**Marking** – Assignment of token count to places



$m : p_1 \ 2p_2 \rightarrow \text{multi-set notation}$

**Executing a transition**  $t \in T$  of Petri net  $N = [P, T, f, w, m_0]$

Change token count of places

**Definition:** Activation and Execution of a Transition

Transition  $t \in T$  of  $N = [P, T, f, w, m_0]$  is *activated* in marking  $m$ , if

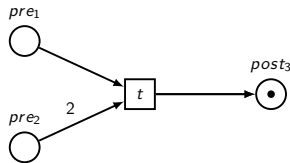
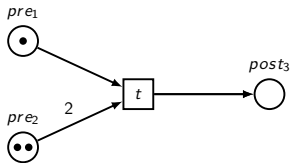
$$\forall p \in P : w(p, t) \leq m(p)$$

Notation:  $m \xrightarrow{t}$

If  $m \xrightarrow{t}$ , *executing*  $t$  in  $m$  leads to marking  $m'$  with

$$\forall p \in P : m'(p) = m(p) + w(p, t) - w(t, p)$$

Notation:  $m \xrightarrow{t} m'$

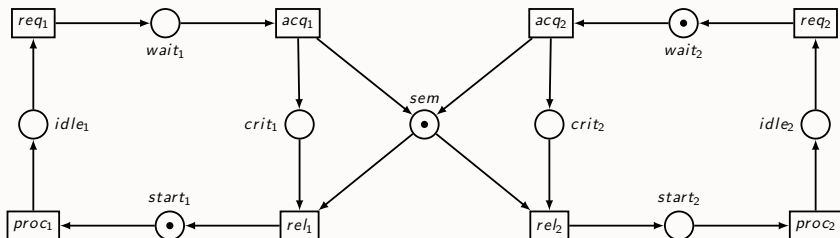


**Example:** Petri net  $N = [P, T, f, w, m_0]$

$P = \{sem, crit_1, start_1, idle_1, wait_1, crit_2, start_2, idle_2, wait_2\}$

$T = \{acq_1, rel_1, proc_1, req_1, acq_2, rel_2, proc_2, req_2\}$

$m_0 : start_1 \ sem \ wait_2$



**State Space** generation of Petri net  $N = [P, T, f, w, m_0]$

*How?*

DFS/BFS exploration – executing activated transitions in a marking

**Definition:** State Space

The *state space*  $R = [V, E]$  of  $N = [P, T, f, w, m_0]$  is a directed, labeled graph, inductively defined as follows:

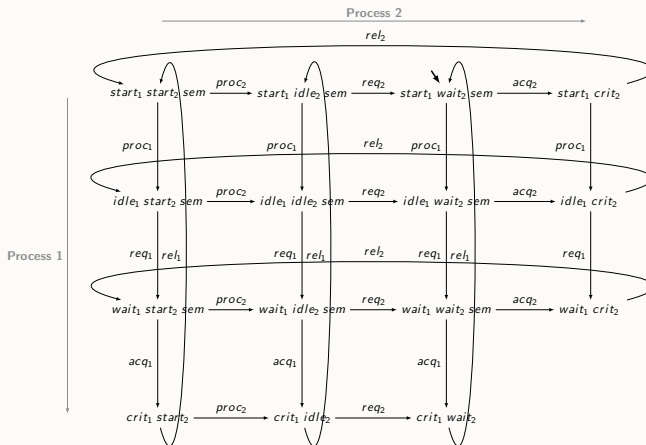
Base:  $m_0 \in V$

Step: If  $m \in V$  and  $m \xrightarrow{t} m'$  for  $t \in T$ , then  $(m, t, m') \in E$  and  $m' \in V$ .

Similar:

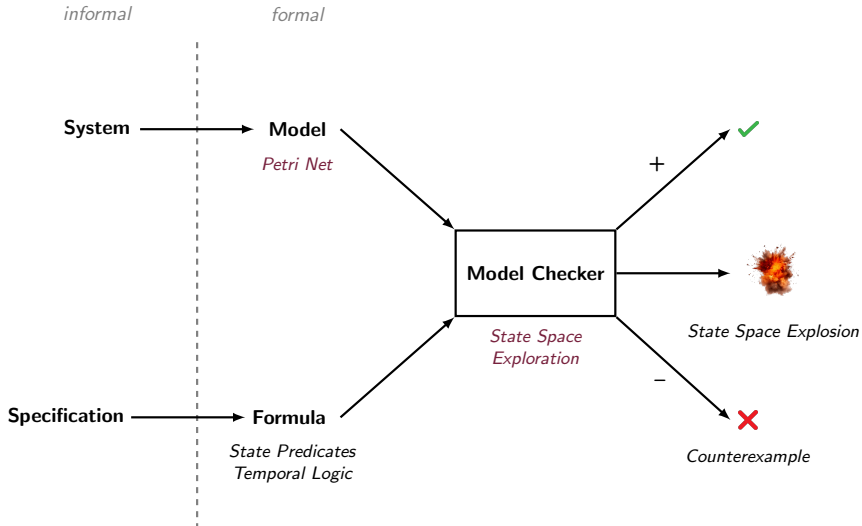
- Labelled Transition System
- Kripke structure [CLM89]

**Example:** State Space  $R = [V, E]$  of Petri net  $N$



with  $|V| = 15$  and  $|E| = 28$

# Model Checking with Petri Nets



### **Note:** Size of the State Space

State space  $R$  can become exponentially large in the size of Petri net  $N$  (or even infinitely big).

→ Avoiding state space explosion by exploiting the Petri net formalism

Strong connection between structure and behavior of Petri nets allows

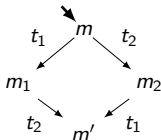
### 1. **State Space Reduction Methods**

→ Minimization of the number of explored states

### 2. **Net Structure Analysis**

→ No state space generation (for specific net classes)

- Idea** Concurrent actions cannot (de)activate each other [EH08]
- Independent in their execution order
  - State reachable by any permutation of concurrent actions



- Goal** Reduce state space by permutations of concurrent actions

- *Stubborn Set Method* [Val91]  
explore only non-concurrent transitions in a marking
- + Enormous reduction esp. in concurrent systems
- Restriction of checkable specifications

**Idea** Symmetrical structures generate symmetrical behavior

**Goal** Reduce state space by symmetrical behavior

- *Symmetry Method* [Sta91] [Sch00]  
Detect symmetric markings by automorphisms of the Petri net and avoid their exploration
- + Enormous reduction esp. in symmetrical systems
- Automorphism = strict mathematical concept allows no deviations  
Restriction of checkable specifications

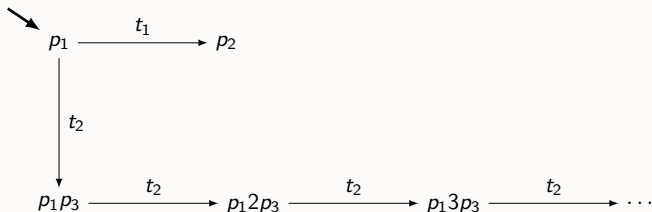
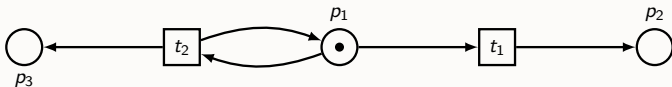
Assume  $N = [P, T, f, w, m_0]$  generates an *infinite* state space

**Idea** Identify and abstract infinite Behavior

**Goal** Finite state space

- *Coverability Graph* [KM69]  
Detect sequences of infinite sequences of states and abstract them
- + Always finite [KM69]
- Coverability = Abstraction → behavior may be concealed  
Restriction of checkable specifications

### Example: Petri Net $N$ with unbounded State Space



Number of Tokens on  $p_3 \rightarrow \infty$

**Coverability Graph** of Petri net  $N = [P, T, f, w, m_0]$

DFS-Exploration

Marking  $m \rightarrow \text{DFS-Stack } \sqcup(m)$

= Stack of markings from  $m_0$  to  $m$

For freshly explored marking  $m$  check:  $\exists m^* \in \sqcup(m) : m^* < m?$

$\rightarrow m$  covers a predecessor marking  $m^*$ ?

$\rightarrow$  behavior of  $m^*$  would be possible in  $m$  as well!

Why?

**Note:** Monotony of Executing Transitions

Assume  $m \xrightarrow{t} m'$  for  $t \in T$  and there exists a marking  $m_1 \geq m$ .

This implies the existence of marking  $m'_1 \geq m'$  with  $m_1 \xrightarrow{t} m'_1$ .

Assume  $\exists m^* \in \sqcup(m) : m^* < m$ .

→ **Pumping sequence**  $\sigma \in T^* : m^* \xrightarrow{\sigma} m$

According to monotony of executing transitions:

→ Pumping sequence  $\sigma$  is executable in  $m$ , leading to marking  $m' > m$

→ Can be repeated infinitely often

→ Infinite sequence of covering markings

Assume place  $p \in P$  with  $m^*(p) < m(p)$ .

→ Token count on  $p$  grows into the infinite

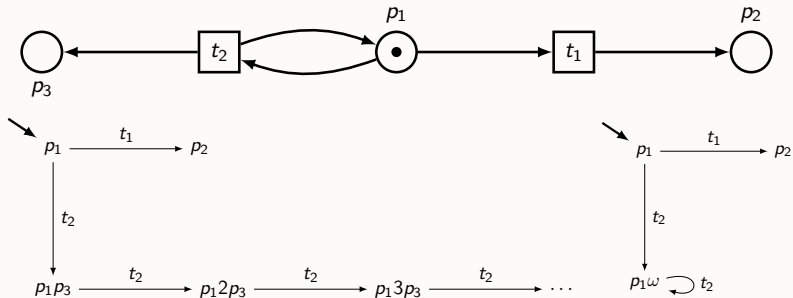
→  **$\omega$ -Abstraction**  $m(p) \leftarrow \omega$

$\omega$  = infinitely many tokens

**Theorem:** Finite Coverability Graph [KM69]

The coverability graph of  $N$  is finite.

## Example: Petri Net $N$ with unbounded State Space



### **3. Modular Petri Nets**

Compositional System = Composition of interacting components

**Principle of Locality** Cause and effect of actions are local

→ Component behavior mostly independent of another

**Principle of Compositionality** [Fre92] System behavior deducible from components behavior and the rules of their interaction

→ System behavior = synchronized component behavior

→ Component behavior restricted in the context of the composition

*Interesting for Model Checking!*

Compositional/Modular Verification – [CLM89] [SG90]

... for Petri Nets – [NM94] [Don01] [CP00] [LM04] [GWW24]  
[GWO24] [ZW24] [WGZW26]

Modeling formalism for compositional systems

Implicit representation of a Petri net by components and interaction rules

**Definition:** Module, Modular Petri Net [GWW24]

A *module* is a Petri net  $N = [P, T, f, w, m_0]$  such that

$T = T_{\text{internal}} \cup T_{\text{interface}}$ , where

- $T_{\text{internal}}$  is a set of internal transitions, and
- $T_{\text{interface}}$  is a set of interface transitions.

A *modular Petri net* is a tuple  $\mathcal{M} = [\mathcal{N}, \mathcal{S}]$ , where

- $\mathcal{N} = \{N_1, \dots, N_\ell\}$  is a set of disjoint modules.
- $\mathcal{S} \subseteq (T_{1|\text{interface}} \cup \{\epsilon\}) \times \dots \times (T_{\ell|\text{interface}} \cup \{\epsilon\})$  is a set of *synchronizations*.

**Example:** Modular Petri Net  $\mathcal{M} = [\mathcal{N}, \mathcal{S}]$

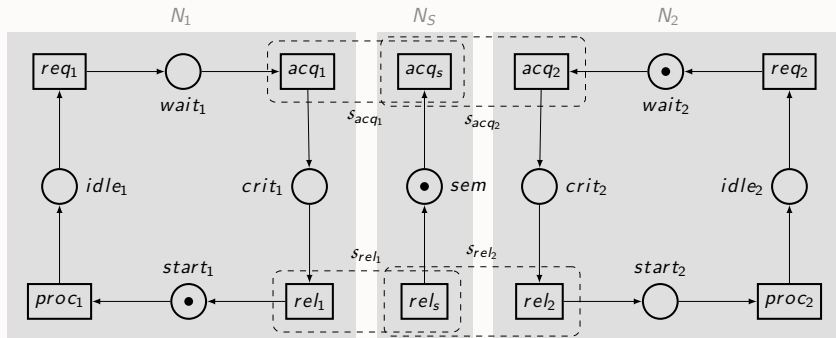
with  $\mathcal{N} = \{N_1, N_2, N_3\}$  and  $\mathcal{S} = \{s_1, s_2\}$ , where

$$s_{acq_1} = [acq_1, acq_3]$$

$$s_{acq_2} = [acq_3, acq_2]$$

$$s_{rel_1} = [rel_1, rel_3]$$

$$s_{rel_2} = [rel_3, rel_2]$$



### Note: Module Behavior

Behavior of module  $N_j \in \mathcal{N}$  is defined locally and independent of the other modules of the modular Petri net  $\mathcal{M} = [\mathcal{N}, \mathcal{S}]$ .

**Local Marking** – Assignment of token count to module places

$$m_j : P_j \rightarrow \mathbb{N}$$

**Executing a module transition**  $t \in T_{j|internal} \cup T_{j|interface}$  of  $N_j \in \mathcal{N}$

Change token count of module places

Activation  $m_j \xrightarrow{t}$  and execution  $m_j \xrightarrow{t} m'_j$  on local markings  $m_j, m'_j$  are defined similar to transitions of a Petri net (cf. Slide 5).

**Note:** Module Composition restricts Module Behavior

No restriction: **Internal transition**

= module-internal behavior

→ Individual execution

Restriction: **Interface transition**

= interaction behavior

→ Execution of  $t \in T_{j|interface}$  only in the context of activated synchronization  $s \in \mathcal{S}$ , where  $s[j] = t$ .

### Marking – Concatenation of local markings

$$m = m_1 \parallel \dots \parallel m_\ell$$

where  $m_j$  is a local marking of module  $N_j \in \mathcal{N}$ .

### Executing a synchronization $s \in \mathcal{S}$

Change number of tokens on the module places globally

#### Definition: Activation and Execution of a Synchronization

Synchronization  $s \in \mathcal{S}$  is *activated* in marking  $m = m_1 \parallel \dots \parallel m_\ell$ , if  $m_j \xrightarrow{s[j]}$  for  $s[j] \in T_{j|interface} \cup \{\epsilon\}$  for each module  $N_j \in \mathcal{N}$ .

*Execution* of activated synchronization  $s \in \mathcal{S}$  in marking  $m = m_1 \parallel \dots \parallel m_\ell$  leads to marking  $m' = m'_1 \parallel \dots \parallel m'_\ell$ , where  $m_j \xrightarrow{s[j]} m'_j$ .

Note that:  $m_j \xrightarrow{\epsilon} m_j$  for every local marking  $m_j$ .

**Modular State Space** generation of modular Petri net  $\mathcal{M} = [\mathcal{N}, \mathcal{S}]$

*How?*

Synchronized exploration of the module behavior

**Definition:** Modular State Space [GWW24]

The *modular state space*  $\mathcal{R} = [\{R_j = [V_j, E_j] \mid N_j \in \mathcal{N}\}, R_S = [V_S, E_S]]$  is a tuple, where

- $R_j = [V_j, E_j]$  is the *local state space* of  $N_j \in \mathcal{N}$   
→ module behavior (internal + interface)
- $R_S = [V_S, E_S]$  is the *synchronization graph*  
→ synchronization behavior

Exhaustively repeat

- i. Exhaustively explore local behavior
  - module internal behavior  $\rightarrow \{R_j = [V_j, E_j]\}$

$\rightarrow$  parallelizable
- ii. Execute a synchronization when activated
  - module interface behavior  $\rightarrow \{R_j = [V_j, E_j]\}$
  - synchronization  $\rightarrow R_S = [V_S, E_S]$

**Base:** abstraction of module behavior between synchronizations  $\rightarrow$  *segment abstraction*

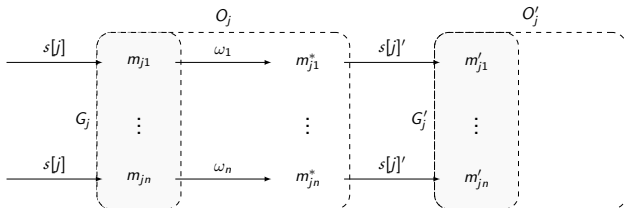
**Definition:** Segment, Generator

A *segment*  $O_j$  is internal-only behavior of a module  $N_j \in \mathcal{N}$  based on set of local markings  $G_j$  (*Generator*).

Formally:  $O_j = \hat{G}_j = \{m'_j \mid m_j \xrightarrow{\sigma} m'_j, m_j \in G_j, \sigma \in T_{j|internal}\}$

$\rightarrow$  may activate further local behavior

Building the **synchronization graph**  $R_S = [V_S, E_S]$ :

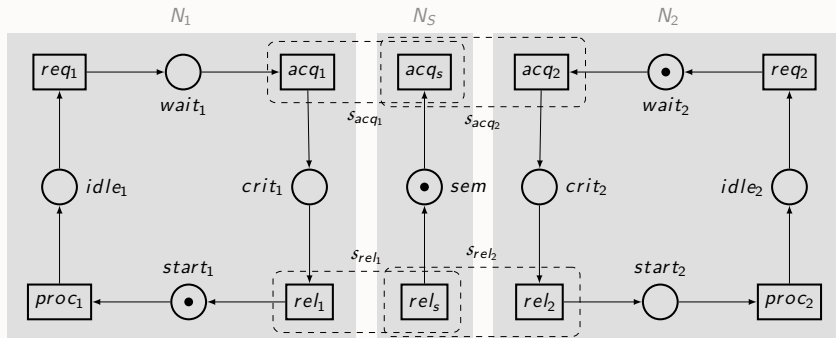


- Collect internal-only behavior of module  $N_j \in \mathcal{N}$  between synchronizations  $\rightarrow$  segment  $O_j$

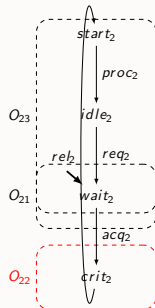
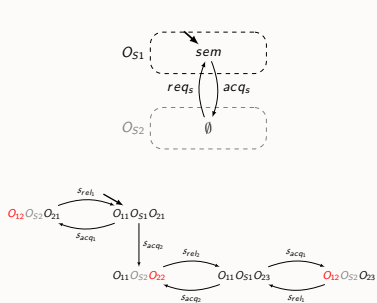
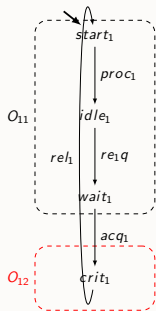
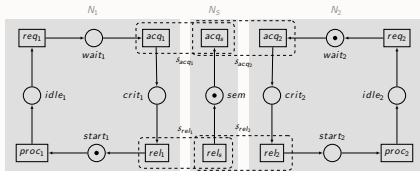
Initial segment:  $O_j = \{\dot{m}_{j0}\}$

- Vertex  $v \in V_S$  :  $\ell$ -tuple of segments  $v = O_1 \dots O_\ell$   
 $\rightarrow$  Abstraction of modular Petri net behavior
- Edge  $(v, s, v') \in E_S$  : Synchronization  $s \in \mathcal{S}$   
 Successor vertex  $v' = O'_1 \dots O'_\ell$ , where  $O'_j = \dot{G}'_j$  for  
 $G'_j = \{m'_j \mid m_j \xrightarrow{s[j]} m'_j, m_j \in O_j\}$

## Example: Modular State Space



## Example: Modular State Space



### Note: Modularization

Every Petri net  $N = [P, T, f, w, m_0]$  can be transformed into a modular Petri net.

How?  $\rightarrow$  [GWO24]

### Theorem: Marking Equivalence [GWW24]

Let Petri net  $N = [P, T, f, w, m_0]$  given in form of a modular Petri net  $\mathcal{M} = [\mathcal{N}, \mathcal{S}]$ .

Let  $R = [V, E]$  be the state space of  $N$  and  $\mathcal{R} = [\{R_j = [V_j, E_j] \mid N_j \in \mathcal{N}\}, R_{\mathcal{S}} = [V_{\mathcal{S}}, E_{\mathcal{S}}]]$  be its modular state space.

It holds that marking  $m \in V$  if and only if there exists a vertex  $O_1 \dots O_{\ell} \in V_{\mathcal{S}}$  such that  $m \in O_1 \times \dots \times O_{\ell}$ .

$\rightarrow$  Modular state space contains the exact set of states of  $N$  😊

Modular state space contains the exact set of states of  $N$

→ State explosion problem remains ☹️

*Why modular Petri nets?*

Exploiting the modular Petri net formalism by

1. **Heterogeneity of the Modules**

→ Module-specific state local state space generation

2. **Instantiation**

→ Reusing local markings

3. **Abstraction**

→ Model checking only on the synchronization graph

to counteract state explosion problem

**Note:** Recall from part 2

- success of state space reduction methods depends on Petri net structure
- state space generation avoidable for distinct Petri net classes only

**Idea** Methods more effective for single modules than for the entire Petri net

**Goal** local state space exploration tailored to the modules  
→ reduced segments

**But** Explore sufficient behavior to maintain marking equivalence to the state space

*Sufficient behavior to maintain marking equivalence?*

**Note:** Requirement for Reduced Segment

A segment must be completely deducible from the reduced segment

More precisely:

Segment  $O_j'$  evolves from executing  $s[j]$  of synchronization  $s \in \mathcal{S}$  in segment  $O_j$



For any synchronization  $s \in \mathcal{S}$ : Local marking  $m_j \in O_j$  with  $m_j \xrightarrow{s[j]}$  should be deducible from reduced segment  $O_j^*$  to guarantee completeness of  $O_j'$  exploration

## Heterogeneity of Modules

Assume segment  $O_j$  with generator  $G_j \subseteq O_j$  of module  $N_j \in \mathcal{N}$ .

Next: Synchronization  $s \in \mathcal{S}$

Build reduced segment  $O_j^*$  if the module is

**conflict-free** with single-element stubborn sets

**concurrency-free**  $O_j^* = G_j$

local marking  $m_j \in O_j$  with  $m_j \xrightarrow{s[j]}$

= Solution of an ILP

NP-complete, but great tool support 😊

= Solution of a maximum flow problem [Wal24]

**acyclic**  $O_j^* = G_j$

+ linear algebra [Sch24]

**unbounded** modular coverability [WGZW26]

**Note:** Recall from Part 2

Unbounded Petri nets  $\rightarrow$  DFS-Stack  $\sqcup(m)$  for marking  $m$  to identify unbounded behavior with pumping sequences:

$$\exists m^* \in \sqcup(m) \text{ with } m^* \xrightarrow{\sigma} m \text{ for } \sigma \in T^* \text{ such that } m^* < m?$$

In the modular state space:

Local state spaces  $\rightarrow$  local parts of the pumping sequence (**local pumping sequences**)

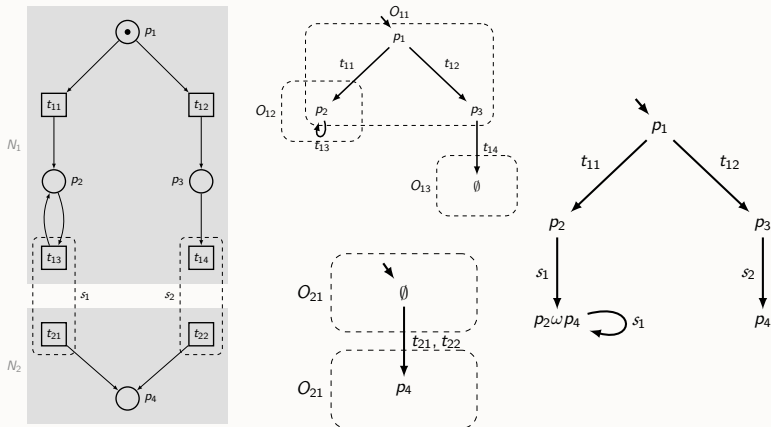
**Good:**

**Theorem:** Projection of Pumping Sequence [WGZW26]

Pumping sequence  $\sigma \in (T_{\text{internal}} \cup \mathcal{S})^* \Rightarrow \exists$  selection  $(\sigma_1, \dots, \sigma_\ell)$ , of local pumping sequences  $\sigma_j \in (T_{j|\text{internal}} \cup T_{j|\text{interface}})^*$  that composes to  $\sigma$ .

**Not So Good:** Not every selection of local pumping sequences composes to an actual pumping sequence ☹

## Example



### **Note:** Composable Local Pumping Sequences

A selection of local pumping sequences needs to align in the succession of occurring synchronizations.

→ Information from the synchronization graph necessary

**Bad:** Combinatorial effort!

Assume synchronization graph vertex  $v = O_1 \dots O_\ell$  and marking  $m \in O_1 \times \dots \times O_\ell$  such that  $m = m_1 \parallel \dots \parallel m_\ell$ .

For  $m_j$ :  $\Sigma_j \subseteq (T_{j|internal} \cup T_{j|interface})^* =$  set of local pumping sequences  
( $\sigma_j \in \Sigma_j$ :  $m_j^* \xrightarrow{\sigma_j} m_j$  such that  $m_j^* < m_j$ )

→ For  $m = m_1 \parallel \dots \parallel m_\ell$ : check if  $(\sigma_1, \dots, \sigma_\ell) \in \Sigma_1 \dots \Sigma_\ell$  aligns in their succession of synchronizations and composes to an actual pumping sequence  $\sigma$  from some marking  $m^* < m$ .

**Good News I:** Segments and generators are useful to overcome combinatorial effort 😊

### Definition: Generator Marking

Let  $v = O_1 \dots O_\ell$  be a synchronization graph vertex, where  $G_j$  is the generator of  $O_j$  for module  $N_j \in \mathcal{N}$ .

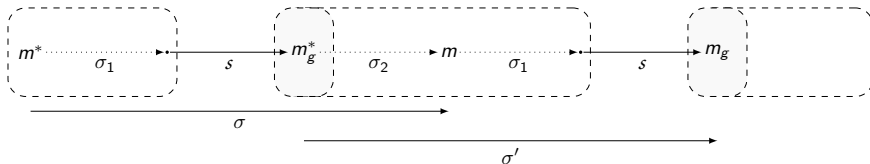
Marking  $m_g \in G_1 \times \dots \times G_\ell$  is a *generator marking*.

### Theorem: Generator Marking Pumping Sequences [WGZW26]

Pumping sequence  $\sigma \in (T_{\text{internal}} \cup \mathcal{S})^*$  such that  $m^* \xrightarrow{\sigma} m$  for  $m^* < m$  implies a pumping sequence  $\sigma' \in T_{\text{internal}} \cup \mathcal{S}$  such that  $m_g^* \xrightarrow{\sigma'} m_g$  for  $m_g^* < m_g$ , where  $m_g^*, m_g$  are generator markings such that

- $m^* \rightarrow m_g^*$
- $m \rightarrow m_g$

Proof Sketch:



Assume

Pumping sequence  $\sigma = \sigma_1 s \sigma_2$  for  $\sigma_1, \sigma_2 \in T_{internal}^*$ ,  $s \in \mathcal{S} : m^* \xrightarrow{\sigma} m$ .  
 $m^* < m$ .

$\Downarrow$

Sequence  $\sigma_1 s$  executable in  $m$ .

$\Downarrow$

Pumping sequence  $\sigma' = \sigma_2 \sigma_1 s$  from  $m_g^*$  to  $m_g$ .

(extendable for sequences with  $> 1$  synchronizations)

**Good News II:** Internal-Only Pumping Sequences directly detectable in the local state space 😊

### Definition: Projection of Sequences

For sequence  $\sigma \in T_{internal}^*$ , let  $\pi_j(\sigma) \in T_{j|internal}^*$  be the *projection* of  $\sigma$  to module  $N_j \in \mathcal{N}$ , such that for transition  $t \in \sigma$ , it holds that  $t \in \pi_j(\sigma)$  if and only if  $t \in T_{j|internal}$ .

### Theorem: Internal-Only Pumping Sequences [WGZW26]

Internal-only pumping sequence  $\sigma \in T_{internal}^* : m^* \xrightarrow{\sigma} m'$  for  $m^* \leq m'$ ,  
iff local pumping sequence  $\pi_j(\sigma) \in T_{j|internal}^* : \pi_j(m^*) \xrightarrow{\pi_j(\sigma)} \pi_j(m')$   
for module  $N_j \in \mathcal{N}$ .

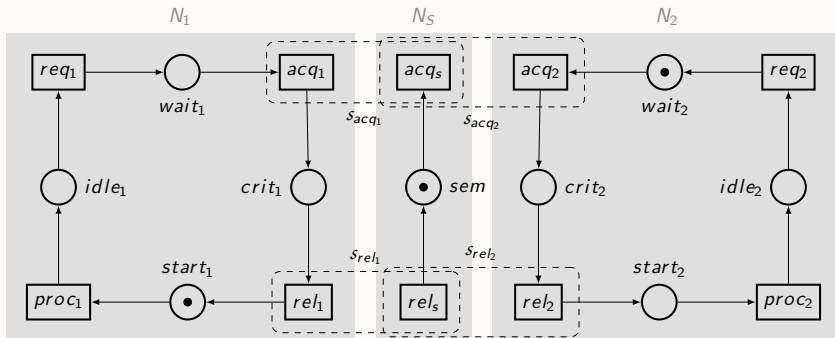
## Instantiation

**Idea** Modules = **Instances** of the same Petri net with different initial markings

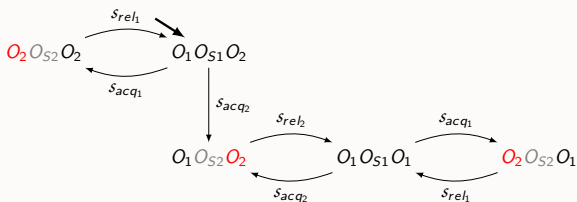
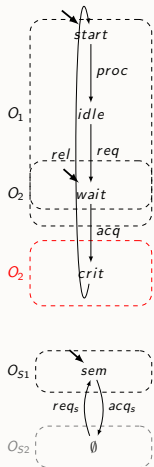
→ more robust than the symmetry method [Sch00]

**Goal** Explore local state space only once

### Example: Instantiation



## Example: Instantiation



→ 11 vertices & 13 edges vs. 15 vertices and 17 edges without instantiation

**Idea** Synchronization graph = Abstraction of the state space  
→ Abstract model checking with the synchronization graph

**Note:** Abstract Model Checking and Behavioral Equivalences

Logic of the formula → necessary *behavioral equivalence* between the system and its abstraction.

State Predicates  
(Propositional Logic)

Linear Time Logic [Pnu77]

Branching Time Logic [BAMP81]

cf. [Gla01]

**State Equivalence**

→ same set of states

**Trace Equivalence**

→ same set of executions

**Bisimulation**

→ same branching structure

*What can we check in the synchronization graph?*

State Predicates ✓  
(Propositional Logic) cf. Thm:Marking Equivalence

Linear Time Logic ✓  
cf. [ZW24]

Branching Time Logic ✗  
→ Refinement necessary!  
cf. [ZW24]

### **Modular Petri Nets**

- = modelling formalism for (compositional) systems based on Petri nets

### **Model Checking for Modular Petri Nets**

Base: Modular state space – marking equivalent to state space

Effective by exploiting

- Heterogeneity of the modules
- Instantiation
- Abstraction

### **Future Work**

- Verification-optimal automatic modularization
- Inclusion of symbolic model checking methods, e.g. BDDs
- State space reduction methods for the synchronization graph

-  Mordechai Ben-Ari, Zohar Manna, and Amir Pnueli, *The Temporal Logic of Branching Time*, Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '81 (Williamsburg, Virginia), ACM Press, 1981, pp. 164–176 (en).
-  E. M. Clarke, E. A. Emerson, and A. P. Sistla, *Automatic Verification of Finite-State Concurrent Systems using Temporal Logic Specifications*, ACM Transactions on Programming Languages and Systems **8** (1986), no. 2, 244–263 (en).
-  E.M. Clarke, D.E. Long, and K.L. McMillan, *Compositional Model Checking*, [1989] Proceedings. Fourth Annual Symposium on Logic in Computer Science, June 1989, pp. 353–362.
-  Søren Christensen and Laure Petrucci, *Modular Analysis of Petri Nets*, The Computer Journal **43** (2000), no. 3, 224–242.

-  Susanna Donatelli, *Kronecker Algebra and (Stochastic) Petri Nets: Is It Worth the Effort?*, Applications and Theory of Petri Nets 2001 (Berlin, Heidelberg) (José-Manuel Colom and Maciej Koutny, eds.), Springer, 2001, pp. 1–18 (en).
-  Jörg Desel and Wolfgang Reisig, *Place/Transition Petri Nets*, Advanced Course on Petri Nets, Springer, 1996, pp. 122–173.
-  Javier Esparza and Keijo Heljanko, *Unfoldings - A Partial-Order Approach to Model Checking*, Monographs in Theoretical Computer Science An EATCS Series, Springer, Berlin, Heidelberg, 2008 (en).
-  Gottlob Frege, *Über sinn und bedeutung*, Zeitschrift für Philosophie und philosophische Kritik **100** (1892), 25–50.

-  R. J. van Glabbeek, *The Linear Time - Branching Time Spectrum I.\* The Semantics of Concrete, Sequential Processes*, Handbook of Process Algebra (J. A. Bergstra, A. Ponse, and S. A. Smolka, eds.), Elsevier Science, Amsterdam, January 2001, pp. 3–99.
-  Julian Gaede, Sophie Wallner, and Judith-Henrike Overath, *Automatic Modularization of Place/Transition Nets*, 2024.
-  Julian Gaede, Sophie Wallner, and Karsten Wolf, *Modular State Spaces - A New Perspective*, Application and Theory of Petri Nets and Concurrency (Cham) (Lars Michael Kristensen and Jan Martijn van der Werf, eds.), Springer Nature Switzerland, 2024, pp. 312–332.
-  Richard M. Karp and Raymond E. Miller, *Parallel Program Schemata*, Journal of Computer and System Sciences **3** (1969), no. 2, 147–195.

-  Timo Latvala and Marko Mäkelä, *LTL Model Checking for Modular Petri Nets*, Applications and Theory of Petri Nets 2004 (Berlin, Heidelberg) (Jordi Cortadella and Wolfgang Reisig, eds.), Springer, 2004, pp. 298–311 (en).
-  M. Notomi and T. Murata, *Hierarchical Reachability Graph of Bounded Petri Nets for Concurrent Software Analysis*, IEEE Transactions on Software Engineering **20** (1994), no. 5, 325–336.
-  Amir Pnueli, *The Temporal Logic of Programs*, 18th Annual Symposium on Foundations of Computer Science (sfcs 1977), October 1977, ISSN: 0272-5428, pp. 46–57.
-  Karsten Schmidt, *How To Calculate Symmetries of Petri Nets*, Acta Informatica **36** (2000), no. 7, 545–590 (en).

-  Marlene Schultz, *Utilization of heterogeneity in modular reachability analysis for petri nets*, AWPN 2024 workshop proceedings, Gesellschaft für Informatik eV, 2024, pp. 10–11.
-  Gil Shurek and Orna Grumberg, *The modular framework of computer-aided verification*, International Conference on Computer Aided Verification, Springer, 1990, pp. 214–223.
-  P. H. Starke, *Reachability Analysis of Petri Nets using Symmetries*, Systems Analysis Modelling Simulation **8** (1991), no. 4-5, 293–303.
-  Antti Valmari, *Stubborn Sets for Reduced State Space Generation*, Advances in Petri Nets 1990 (Berlin, Heidelberg) (Grzegorz Rozenberg, ed.), Springer, 1991, pp. 491–515 (en).

-  Sophie Wallner, *State machines (in modular petri nets)*, AWPN 2024 workshop proceedings, Gesellschaft für Informatik eV, 2024, pp. 7–9.
-  Sophie Wallner, Julian Gaede, Lukas Zech, and Karsten Wolf, *Coverability abstraction for the modular state space*, International Conference on Applications and Theory of Petri Nets and Concurrency, Springer, 2026, pp. 312–332.
-  Lukas Zech and Karsten Wolf, *Verifying Temporal Logic Properties in the Modular State Space*, Application and Theory of Petri Nets and Concurrency (Cham) (Lars Michael Kristensen and Jan Martijn van der Werf, eds.), Springer Nature Switzerland, 2024, pp. 333–354 (en).